



BITS Pilani
K K Birla Goa Campus

ADDRESSING MODES

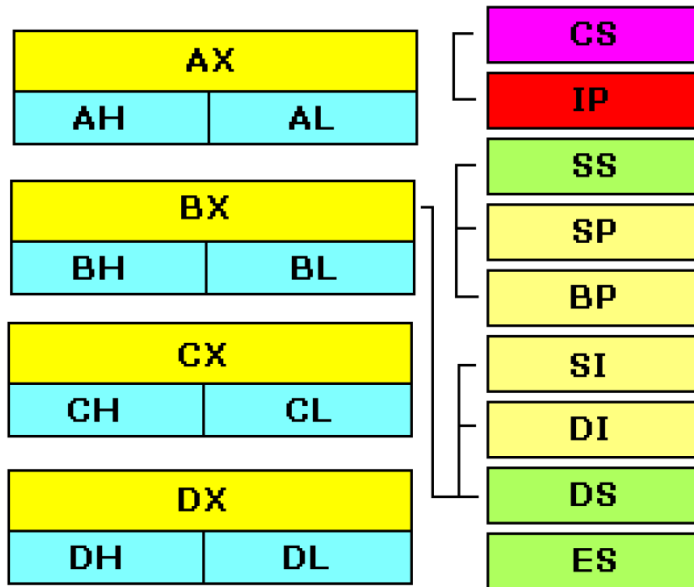
MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS

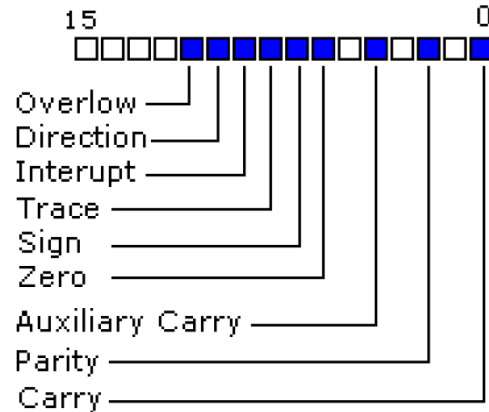


Registers in 8086

Central Processing Unit (or CPU)



Arithmetic & Logical Unit (or ALU)



Example of Overflow Flag

The **Overflow Flag** is set to 1 if the result of an operation is **too large (positive)** or **too small (negative)** to fit into the destination register using 2's complement representation.

- **OF = 1** if there is a carry into the MSB but no carry out, or vice versa.
- **OF = 0** if the carries are the same.

Overflow Conditions

For ADD Instructions

- Positive + Positive $\rightarrow$ Negative
- Negative + Negative $\rightarrow$ Positive

For SUB instruction

- Positive - Negative $\rightarrow$ Negative
- Negative - Positive $\rightarrow$ Positive

Overflow occurs when:

- **Operands have SAME sign**
Result has OPPOSITE sign

Example: 8-bit Signed Addition

MOV AL, 7FH ; AL = 127 (largest positive 8-bit signed number)

ADD AL, 01H ; AL = 128 (out of range for signed 8-bit)

-128 - 1 = -129 → out of range

Result wraps to 7FH (+127)

OF = 1

Example: Positive + Positive → Negative (Overflow)

MOV AL, 64H ; AL = 100

ADD AL, 64H ; AL = 100

100 + 100 = 200

Signed 8-bit range: -128 to +127

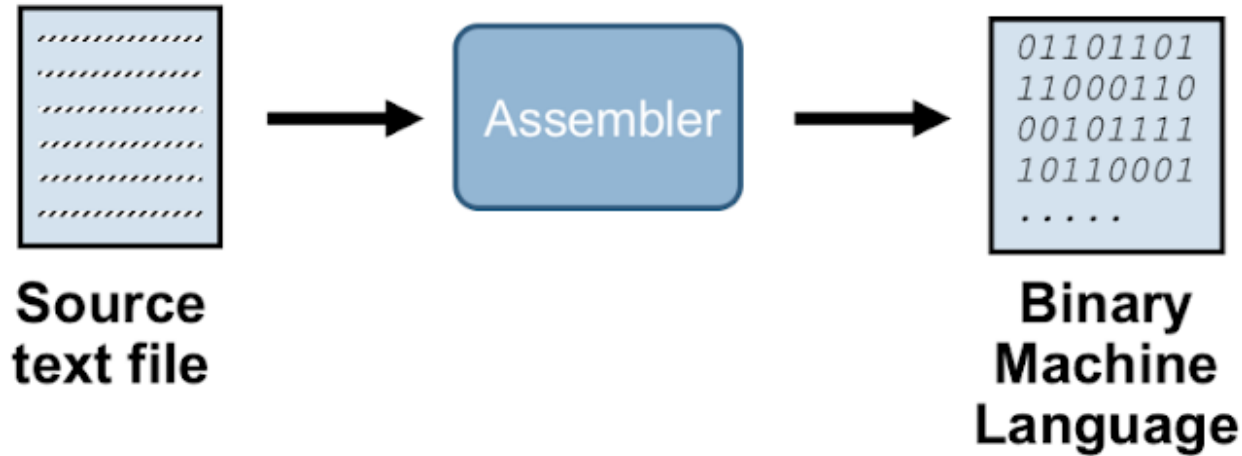
200 is **out of range** → wraps to C8H (-56)

Machine Language

- A sequence of binary codes for the instructions you want the microprocessor to execute
- What form a machine can understand?
- Why is it difficult to write machine code?
 - Memorizing thousands of binary instruction codes
 - An error can occur when working with long series of 1's and 0's
 - Can hexadecimal representation help?
 - Thousands of instructions code to cope with

Assembler

- Reads the file of assembly language program and generates the correct binary code for each instruction

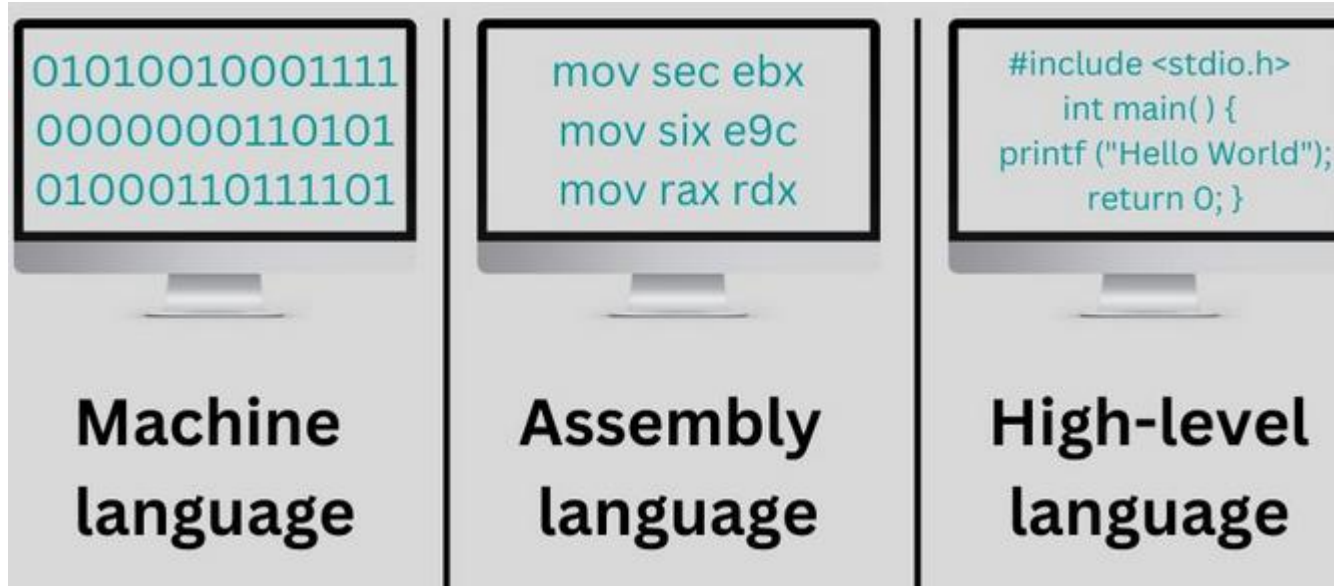


Assembly Language

- Assembly language program is translated to machine language, loaded in memory and run.
- Use two-,three- or four-letter mnemonics to represent each instruction type e.g. ADD, SUB, OR, XOR
- Assembly language instructions are written with four fields

LABEL FIELD	OP CODE FIELD	OPERAND FIELD	COMMENT FIELD
NEXT:	ADD	AL,07H	;ADD Correction factor

Machine <- High Level Language



Addressing Modes

- Different ways in which a processor can access data are referred to as its addressing modes.
- Why we need addressing modes?

Choosing the appropriate addressing mode can **lead to more compact and faster code execution, optimizing program efficiency and performance.**

Addressing modes **determine how operands are located**, whether in registers or memory.

By allowing operations on variables or constants located at different memory addresses or registers, addressing modes **support the creation of versatile and powerful instructions.**

Addressing modes **facilitate the manipulation of arrays and data structures.**

Addressing Modes

- How do you find your friend's house?

House number → ***Direct addressing***

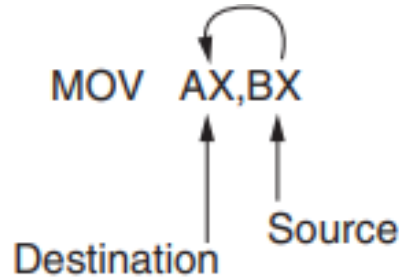
Landmark + turns → ***Indexed***

Friend sends location → ***Indirect***

You already know his room → ***Register***

MOV Instruction

- MOV instruction is a very common and flexible instruction
- Provides a basis for the explanation of the data-addressing modes.



- Transfers the word contents of the source register (BX) into the destination register (AX).
- The source and destination are often called operands.

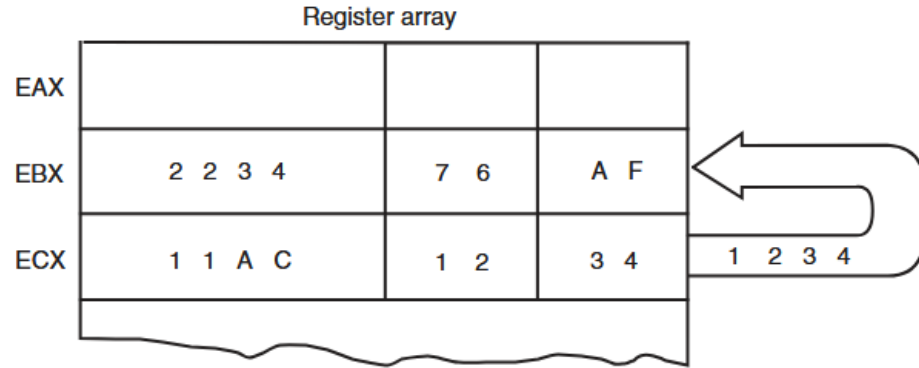
Register Addressing

- Transfers a copy of a byte or word from the source register or contents of a memory location to the destination register or memory location

e.g. MOV CX, DX

- copies the word-sized contents of register DX into register CX
- CX= 2A84H DX=4971H
- After instruction execution, DX= 4971H CX=4971H

Register Addressing: mov bx,cx



0000	8B C3	MOV AX,BX	;copy contents of BX into AX
0002	8A CE	MOV CL,DH	;copy contents of DH into CL
0004	8A CD	MOV CL,CH	;copy contents of CH into CL
0006	66 8B C3	MOV EAX,EBX	;copy contents of EBX into EAX
0009	66 8B D8	MOV EBX,EAX	;copy contents of EAX into EBX
000C	66 8B C8	MOV ECX,EAX	;copy contents of EAX into ECX
000F	66 8B D0	MOV EDX,EAX	;copy contents of EAX into EDX
0012	8C C8	MOV AX,CS	;copy CS into DS (two steps)
0014	8E D8	MOV DS,AX	
0016	8E C8	MOV CS,AX	;copy AX into CS (causes problems)

Immediate Addressing

- Transfers the source, an immediate byte, word, doubleword, or quadword of data, into the destination register or memory location.

e.g. `MOV AL, 22H`

- Copies a byte-sized 22H into register AL.

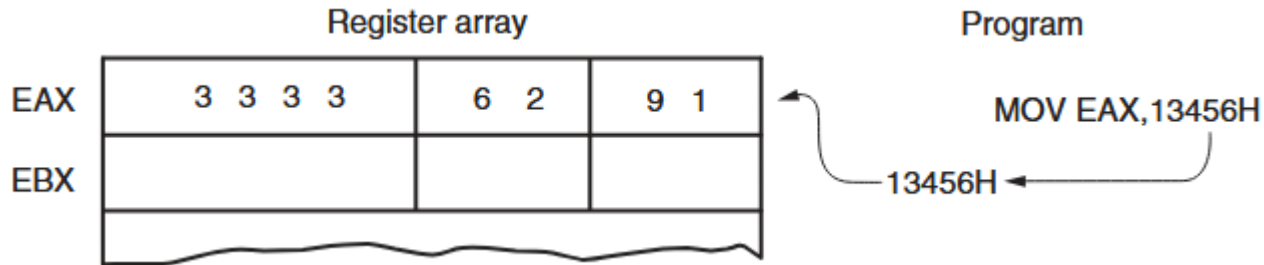
e.g. `MOV CX, 437BH`

- Copied in CH and CL memory location

e.g. `MOV EBX, 12345678H` [In 80386]

- copies a doubleword-sized 12345678H into the 32-bit-wide EBX register

Immediate Addressing:



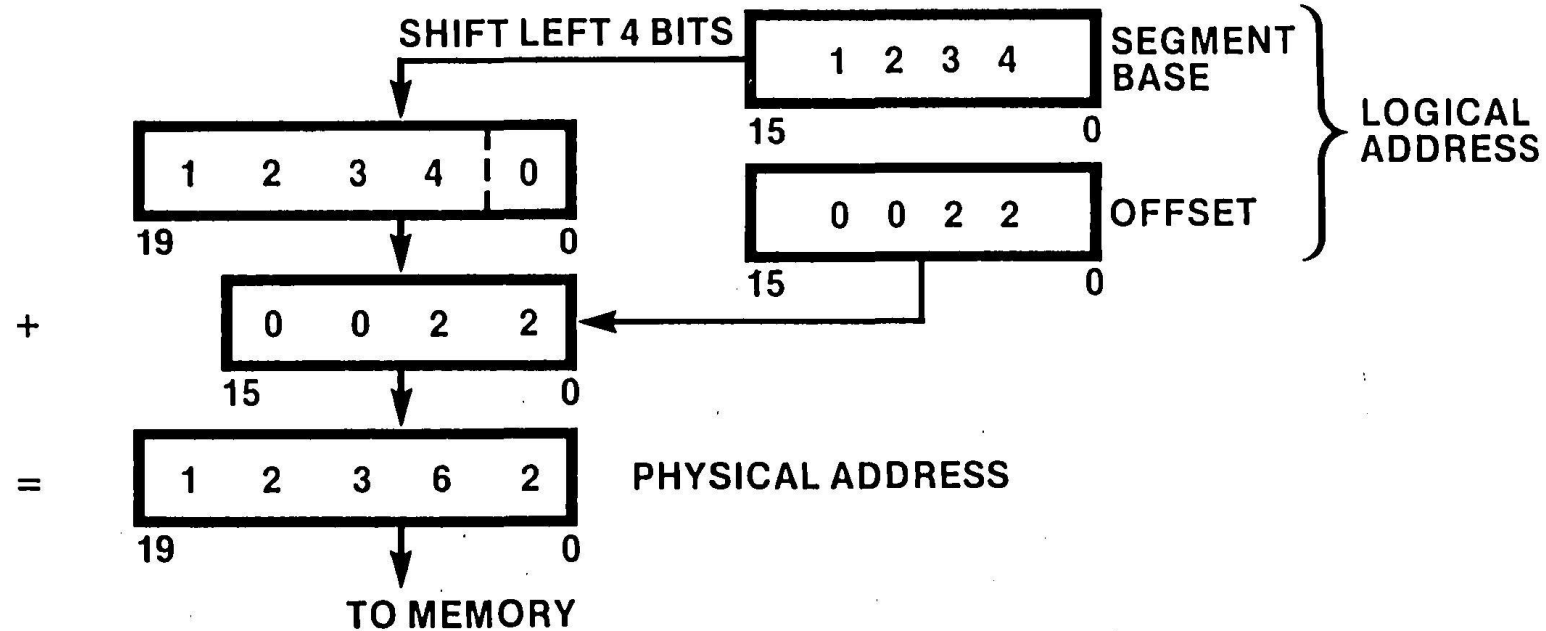
```
.MODEL TINY           ;choose single segment model
.CODE                 ;start of code segment
.STARTUP              ;start of program

MOV AX,0              ;place 0000H into AX
MOV BX,0              ;place 0000H into BX
MOV CX,0              ;place 0000H into CX

MOV SI,AX              ;copy AX into SI
MOV DI,AX              ;copy AX into DI
MOV BP,AX              ;copy AX into BP

.EXIT                 ;exit to DOS
END                   ;end of program
```

Accessing Data in Memory



Direct Addressing

- Moves a byte or word between a memory location addressing and a register.
- The instruction set does not support a memory-to memory transfer, except with the MOVS instruction.

e.g. `MOV CX, LIST`

- copies the word-sized contents of memory location LIST into register CX

e.g. `MOV BL, [437AH]`

BIU calculates physical address

Consider DS=2000

Direct Addressing

e.g. MOV BX, [437AH]

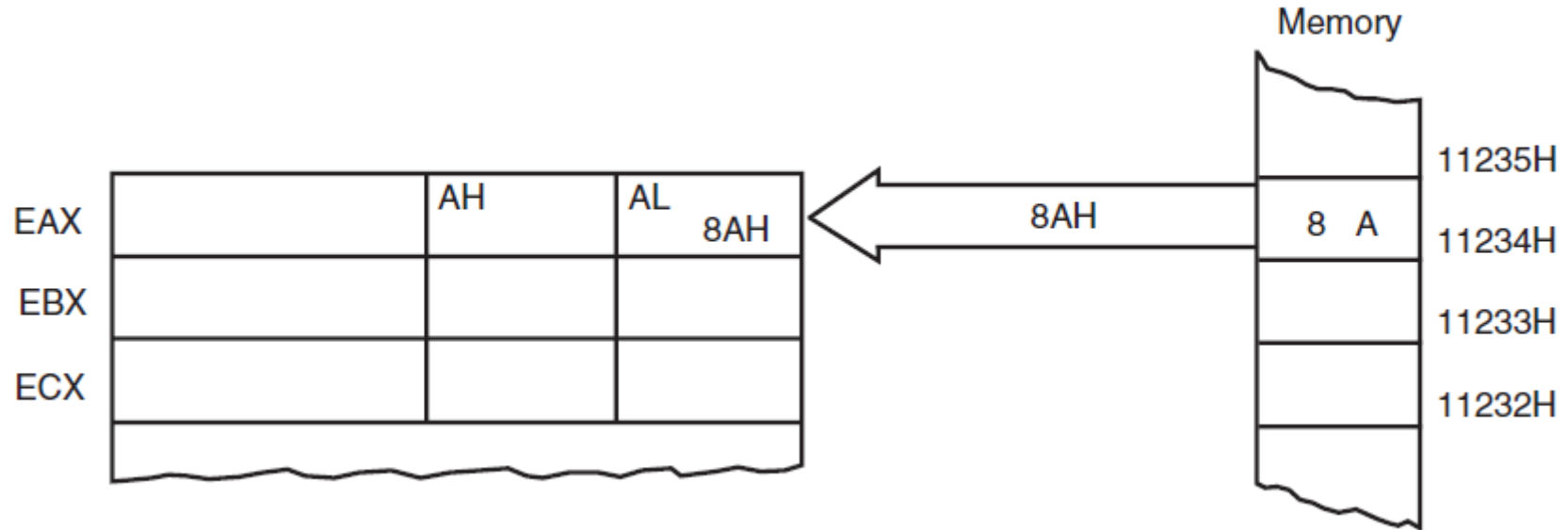
- Each memory address in 8086 represents a byte in storage
- Word must come from two memory locations

BL= Data at 437AH

BH= Data at 437BH

Low Byte - low address High Byte – high address

Direct Addressing : mov AL, [1234H]



Examples

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV AL,NUMBER	8 bits	Copies the byte contents of data segment memory location NUMBER into AL
MOV AX,COW	16 bits	Copies the word contents of data segment
MOV NEWS,AL	8 bits	Copies AL into byte memory location NEWS
MOV THERE,AX	16 bits	Copies AX into word memory location THERE

Register Indirect Addressing

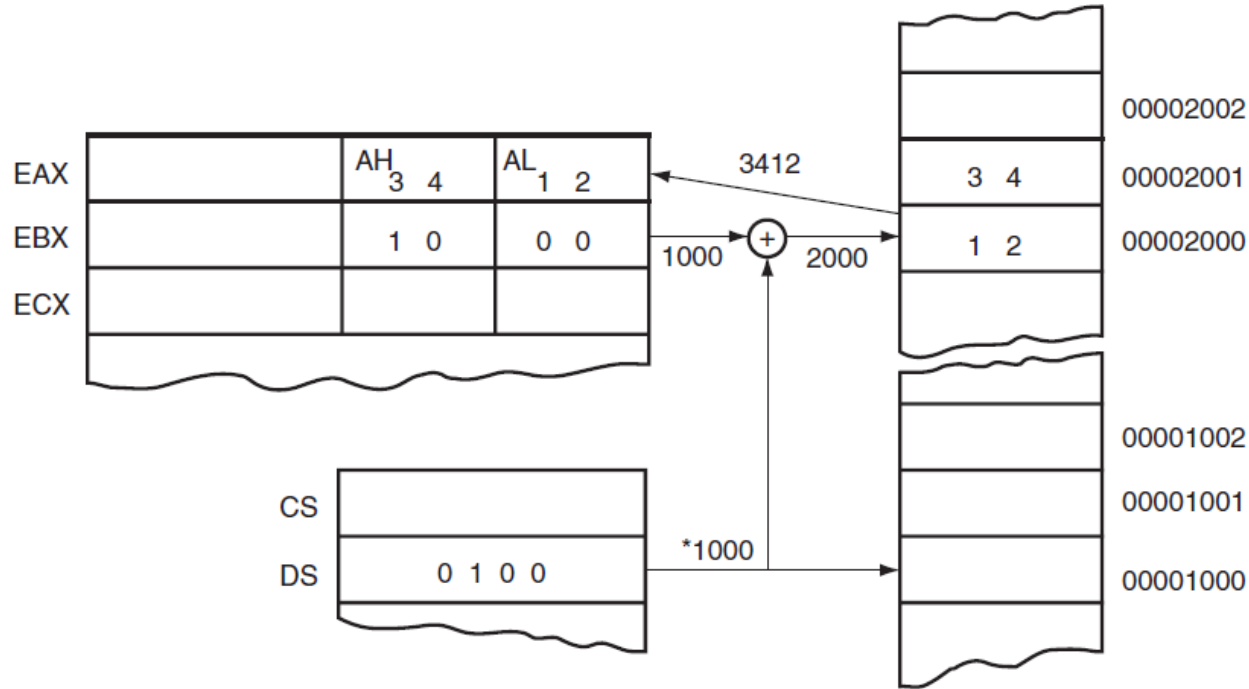
- Transfers a byte or word between a register and a memory location addressed by an index or base register.
- The index and base registers are BP, BX, DI and SI.

e.g. `MOV AX, [BX]`

`BX=1000H & DS=0100H`

- copies the word-sized data from the data segment offset address indexed by BX into register AX.

Register Indirect Addressing : `mov AX, [BX]`



Register Indirect Addressing

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV CX,[BX]	16 bits	Copies the word contents of the data segment memory location addressed by BX into CX
MOV [BP],DL*	8 bits	Copies DL into the stack segment memory location addressed by BP
MOV [DI],BH	8 bits	Copies BH into the data segment memory location addressed by DI
MOV [DI],[BX]	—	Memory-to-memory transfers are not allowed except with string instructions
MOV AL,[EDX]	8 bits	Copies the byte contents of the data segment memory location addressed by EDX into AL
MOV ECX,[EBX]	32 bits	Copies the doubleword contents of the data segment memory location addressed by EBX into ECX
MOV RAX,[RDX]	64 bits	Copies the quadword contents of the memory location address by the linear address located in RDX into RAX (64-bit mode)

Base-plus-index Addressing

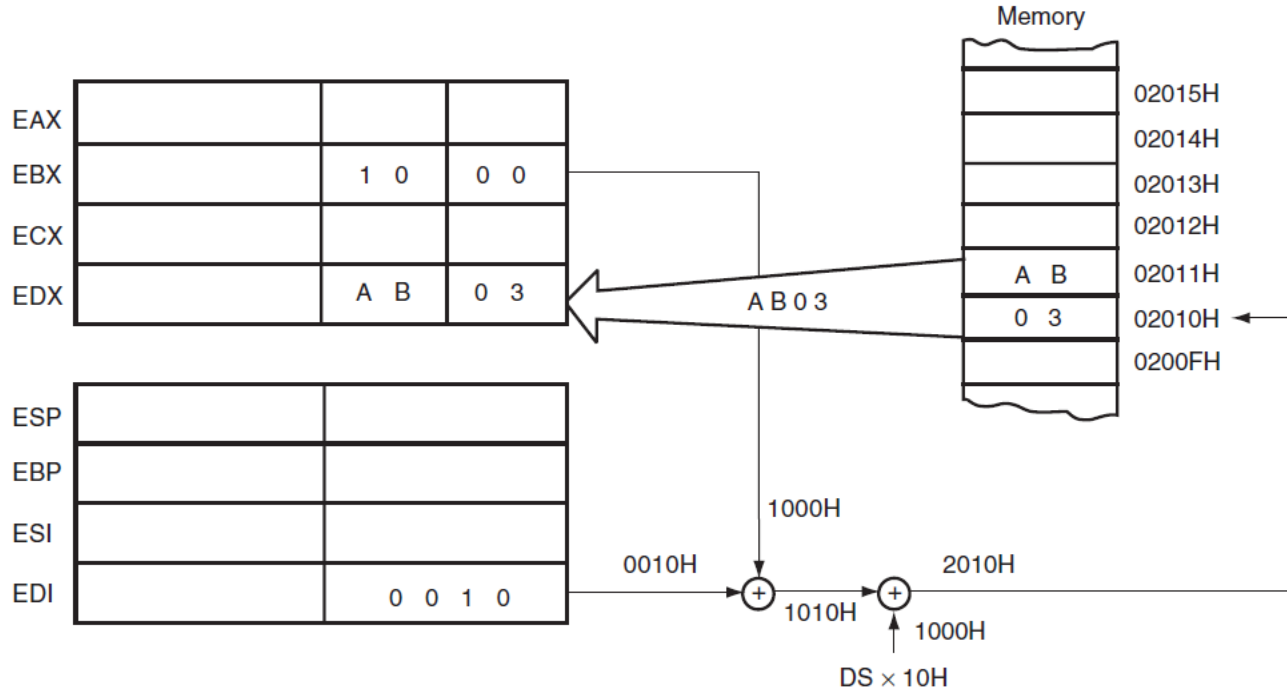
- Transfers a byte or word between a register and the memory location addressed by a base register (**BP or BX**) plus an index register (**DI or SI**).

The base register often holds **the beginning location of a memory array**, whereas the index register holds **the relative position of an element** in the array.

e.g. `MOV [ BX+DI ], CL`

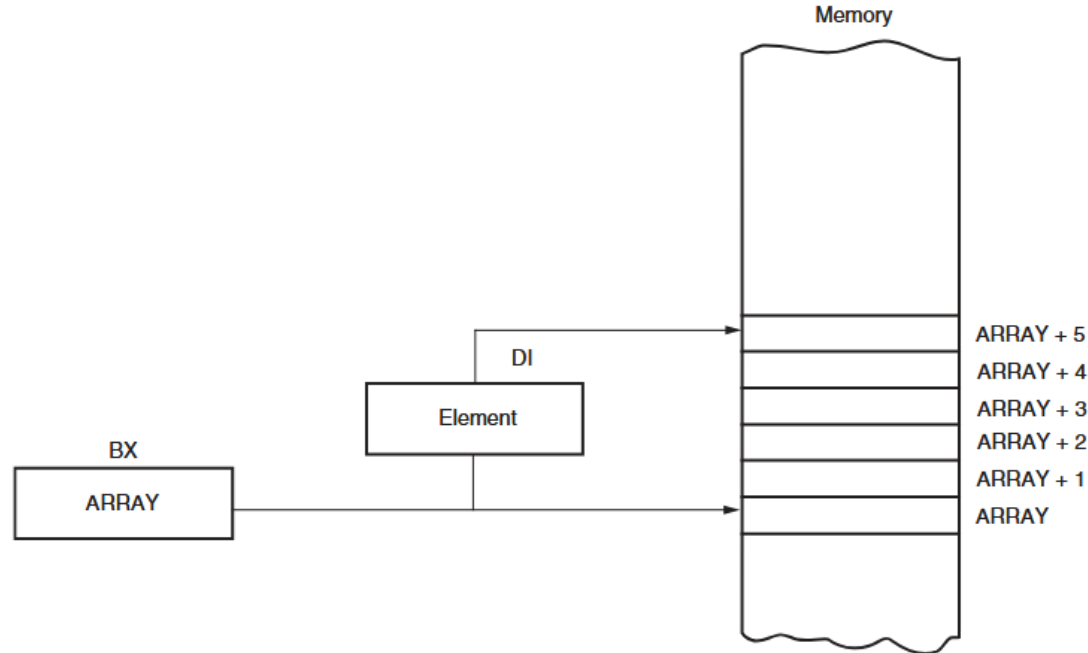
- copies the byte-sized contents of register CL into the data segment memory location addressed by BX plus DI.)

Base-plus-index Addressing: `mov DX,[BX+DI]`



Base-plus-index Addressing: `mov DX,[BX+DI]`

- A major use of the base-plus-index addressing mode is to address elements in a memory array



Base-plus-index Addressing

<i>Assembly Language</i>	<i>Size</i>	<i>Operation</i>
MOV CX,[BX+DI]	16 bits	Copies the word contents of the data segment memory location addressed by BX plus DI into CX
MOV CH,[BP+SI]	8 bits	Copies the byte contents of the stack segment memory location addressed by BP plus SI into CH
MOV [BX+SI],SP	16 bits	Copies SP into the data segment memory location addressed by BX plus SI
MOV [BP+DI],AH	8 bits	Copies AH into the stack segment memory location addressed by BP plus DI
MOV CL,[EDX+EDI]	8 bits	Copies the byte contents of the data segment memory location addressed by EDX plus EDI into CL
MOV [EAX+EBX],ECX	32 bits	Copies ECX into the data segment memory location addressed by EAX plus EBX
MOV [RSI+RBX],RAX	64 bit	Copies RAX into the linear memory location addressed by RSI plus RBX (64-bit mode)

Example

```
.MODEL SMALL
.DATA
ARRAY DB 16 DUP(?)

DB 29H
DB 20 dup(?)

.CODE
.STARTUP

MOV BX,OFFSET ARRAY
MOV DI,10H
MOV AL,[BX+DI]
MOV DI,20H
MOV [BX+DI],AL

.EXIT
END
```

Example: Moves element 10H into array element 20H

```
.MODEL SMALL                ;select small model
.DATA                      ;start data segment
ARRAY DB 16 DUP(?)         ;setup array of 16 bytes

DB 29H                      ;element 10H
DB 20 dup(?)

.CODE                      ;start code segment
.STARTUP

MOV  BX,OFFSET ARRAY        ;address ARRAY
MOV  DI,10H                 ;address element 10H
MOV  AL,[BX+DI]             ;get element 10H
MOV  DI,20H                 ;address element 20H
MOV  [BX+DI],AL             ;save in element 20H

.EXIT                      ;exit to DOS
END                          ;end program
```



BITS Pilani
K K Birla Goa Campus

THANK YOU

